

Industrial Functional Programming

Melinda Tóth, István Bozó



Dept. Programming Languages and Compilers
Eötvös Loránd University, Budapest, Hungary

Contents

- 1 Eunit
- 2 Common Test
- 3 Proper/QuickCheck
- 4 Others

EUnit

- set of tests
- set up the necessary environment
- run the tests
- collect the results
- cleanup

Test files

- `-include_lib("eunit/include/eunit.hrl").`
- `*_test` functions are tests
- Simple tests:
`a_test() -> length([1,2]).`
`b_test() -> [] = length([]).`
- Using `?assert`:
`c_test() -> ?assert(length([]) == []).`
- Running tests: `m:test()`
`eunit:test(m), eunit:test({inparallel, m})`

Test data

- `*_test_()` functions are test generators

- Using `?assert`:

```
d_test_() -> fun() -> ?assert(length([]) == []) end.
```

- Using `?_test`:

```
d2_test_() -> ?_test(?assert(length([]) == [])).
```

- Using `?_assert`:

```
d3_test_() -> ?_assert(length([]) == []).
```

Complex Tests

```
my_test_() ->  
  [?_assert(length([]) == []),  
   ?_test(?assert(length([]) == [])),  
   ?_assert(length([]) == 0)].
```

Test objects

- A zero-argument function:
`fun() -> ... end, fun myfun/0`
- Tuple: {test, Mod, Fun} or {Mod, Fun}
- Pair: {LineNum, TestObj} – result of `?_test`
- Test set: list of test objects (deep list)
- Optional title: “My test”, T

Running tests

- **Primitives:**

`{module, M}, {application, A}, Path, {dir, D}`

- **Control:**

`{spawn, T}, {inparallel, T}, {timeout, Time, T}`

Fixtures

- `{setup, Setup, Tests}`
- `{setup, Setup, Cleanup, Tests}`
- `{setup, Where, Setup, Tests}`
- `{setup, Where, Setup, Cleanup, Tests}`
- `{foreach,...}, { node, Node, ...}`

Complex Tests

```
higher_order_test_() ->
  {"Higher Order Tests",
   [
     {"Setup Test",
      {setup,
       fun () -> io:format("Eleje~n") end,
       fun (ok) -> io:format("Vege~n") end,
       fun (X) ->
         [{"1st", ?_assert(X == ok)},
          {"2nd", ?_assert(X == ok)},
          {"3rd", ?_assert(X == ok)}]
        end}
      },
     {"Foreach Test",
      {foreach,
       fun () -> io:format("Eleje~n"), 3 end,
       fun (3) -> io:format("Vege~n") end,
       [fun (R) -> {"1st", ?_assert(R == 3)} end,
        fun (R) -> {"2nd", ?_assert(R == 3)} end,
        fun (R) -> {"3rd", ?_assert(R == 3)} end]
      }
     ]
  }.
```

Assert macros

- `assert`, `assertNot`
- `assertMatch`, `assertNotMatch`
- `assertEqual`, `assertNotEqual`
- `assertException`
- `assertError`, `assertExit`, `assertThrow`

Using rebar3

- `rebar3 eunit`
- `{eunit_tests, [{m,a_test}]}`.

```
{eunit_first_files, [...]}.  
{eunit_opts, [...]}.  
{eunit_tests, [...]}.  

```

Common Test

- From function/module testing to “System Testing”:
- test cases (function)
- test suites (module, *_SUITE.erl)
- test object directory (folder)
- test root (folder)

Structure

- rootdir -> objectdir -> mod_SUITE.erl
- mod_SUITE_data folder
- logging directory can be set: priv dir + HTML files
- *_SUITE.erl

```
-include_lib("common_test/include/ct.hrl").  
-export([all/0]).  
-export([test1/1, test2/1]).
```

Running tests

- `ct:run_tests/1`
- Log files are generated
- Options: http://erlang.org/doc/man/ct.html#run_test-1

```
ct:run_test([suite, first_SUITE ])).  
ct:run_test([suite, first_SUITE ],  
             {logdir, "./log"})).
```

Config

- proplist
- possible to initialize and modify between test cases
- get values: `?config(Key, Config)`

```
test1(_) ->  
  1 = 2.
```

```
test2(Config) ->  
  io:format("CF: ~p~n", [Config]), %% ct:pal(...)  
  0 = length([]).
```

Using rebar3

- `rebar3 ct`
- `{ct_opts, [{dir, "../apps/chat/test"}]}`.

```
{ct_first_files, [...]}.  
{ct_opts, [...]}.  
{ct_readable, true | false}.
```

Organising test cases

- Hierarchically grouping test cases:

```
groups() -> [{GName, GProp, GMembers}]
```

- Name: atom

- Members: [TestName | {group, GName}]

- Properties:

```
[shuffle, sequence, parallel, {repeat, N},  
{repeat_until_any(all)_fail, N}, – forever  
{repeat_until_any(all)_succeed, N}]
```

- [] – all tests are evaluated in after each other

Setup and teardown functions

- Need to be exported!
- `init_per_testcase(TestName, Config)` -
evaluated in the same process
- `end_per_testcase(TestName, Config)` - **evaluated**
in the same process
- `init_per_group(GroupName, Config)`
- `end_per_group(GroupName, Config)`
- `init_per_suite(Config)`
- `end_per_suite(Config)`

Test specifications

- A proplist in a text file, for example: `chat.test`
- Using it: `ct:run_test([spec, "chat.test"])`.

```
{include, [Dirs]}, {logdir, Dir},  
{suites, Dir, Suites},  
{skip_suites, Dir, Suites, Comment},  
{groups, Dir, Suite, Groups},  
{groups, Dir, Suite, Groups, {cases,Cases}},  
{skip_groups, Dir, Suite, Groups, Comment},  
{skip_groups, Dir, Suite, Groups, {cases,Cases}, Comment},  
{cases, Dir, Suite, Cases},  
{skip_cases, Dir, Suite, Cases, Comment},  
{alias, Alias, Dir}
```

Testing on distributed nodes

- `ct_slave` and `ct_master`
- Start a distributed node:

```
ct_master:run(TestSpecFile).
```

```
{node, NodeAlias, NodeName},  
{init, NodeAlias, [{node_start, [Opts]}]}
```

Opts:

```
{monitor_master, true | false},  
{boot_timeout, Seconds},  
{init_timeout, Seconds},  
{startup_timeout, Seconds},  
{kill_if_fail, true | false},  
{erl_flags, String},  
{startup_functions, [{M,F,A}]}
```

++

```
{*, Nodes, ...}
```

Calling Eunit Tests

```
eunittest(_Config) ->  
  ok = eunit:test(Tests).
```

Other Test Callbacks

Also possible to test over Telnet, SNMP, FTP, SSH

Proper

- <http://proper.softlab.ntua.gr/>
- Property based testing on randomly generated test data
- `-include_lib("proper/include/proper.hrl").`
- Optionally add to code path, or copy proper/ebin to erlang/lib
- Run: `proper:quickcheck(mod:some_property()).`

Properties using macros

- `?FORALL(Pattern, TypeGen, Prop)`
- `?IMPLIES(Cond, Prop)`
- `?SUCHTHAT(Pattern, TypeGen, Cond)`
- `?LET(Pattern, TypeGen, ResExpr)`

Examples

```
prop_delete() ->
  ?FORALL({X,L}, {integer(),list(integer())},
    not lists:member(X, delete(X,L))).

prop_delete_more() ->
  numtests(1000,
    ?FORALL({X,L}, {integer(),list(integer())},
      not lists:member(X, delete(X,L)))).

prop_delete_with_stats() ->
  ?FORALL({X,L}, {integer(),list(integer())},
    collect(lists:member(X,L),
      not lists:member(X, delete(X,L)))).

prop_delete_only_interesting1() ->
  ?FORALL({X,L}, {integer(),list(integer())},
    ?IMPLIES(lists:member(X,L),
      not lists:member(X, delete(X,L)))).
```

Examples

```
list_and_elem1() ->
  ?SUCHTHAT({X,L}, {integer(),list(integer())},
    lists:member(X,L)).

prop_delete_only_interesting2() ->
  ?FORALL({X,L}, list_and_elem1(),
    not lists:member(X, delete(X,L))).

list_and_elem2() ->
  ?LET(L, non_empty(list(integer())), {oneof(L),L}).

prop_delete_only_interesting3() ->
  ?FORALL({X,L}, list_and_elem2(),
    not lists:member(X, delete(X,L))).
```

Others

- Meck
- QuickCheck